[9/25/2025]

Project Plan

[31] [Scottdale Fire Department AI BI Dashboard]

PREPARED BY: TEAM [31]

[Michael Krasnik, Damion Dray, Andrew Tellez, Leroy Sturdivent, Zachary Alexander]

Table of Contents

PROJECT DESCRIPTION

OVERVIEW (ABET-2):

This Project involves the development of a dashboard system that is designed to visualize KPIs of the fire department for its Fire and EMS operations using uploaded CSV/Excel files. The system will integrate generative AI into its pipeline through a team license to ensure no proprietary information is used to train OpenAI's models. The system will support monthly data updates with the ability to append new information to an existing dataset spanning 24-36 months of historical data. A focus on this project is the transparency of the system. We want users to be able to not only view the data in its visualized form, but also understand AI conclusions from this data by interpreting suggested conclusions or querying for answers.

GLOBAL TRENDS (EM@FSE-e)

The growth of AI in the global economy is asking for its integration into the public safety sector. It can assist in many fields including operational efficiency. Many agencies are adopting BI dashboards for fire and EMS data, however those solutions are high cost and often too informative for the users. As it was described to us, the end user doesn't need extra information, only what is asked for in the most basic and straightforward way. At the same time, there is an increasing emphasis on explainable and customizable AI, which provides stakeholders with insights in language and formats tailored to their preferences.

MARKET ANALYSIS (EM@FSE-κ)

The market for Fire and EMS reporting is currently filled by either expensive software platforms that provide too much irrelevant functionality and information or manual reporting processes. Commercial vendors sell solutions with BI intelligence with features such as heatmaps, GIS maps, KPI dashboards or automated summaries. These tools work, but are often too expensive which isn't a realistic undertaking to many municipalities. On the other side, some departments turn to manual reporting processes which are prone to human error and unnecessary time spent. This approach does not provide real time insights.

Our project provides a better option by combining both approaches:

- Lower cost: Using licenses for services provided by ASU (if needed) + simplifying deliverables of visualization tools and catering to exactly what's needed rather than selling what's already built and catering.

- Time savings: Automates data uploads and report generation, cutting down on manual work.

- Flexibility: Adaptive dashboards, generative AI explanations as well as PDF summaries available for export.

- Scalability: Can handle monthly updates and years of data.

SECURITY CONSIDERATIONS (SER-2)

Data security is an important part of this project. Before the datasets were shared, sensitive details such as GPS coordinates, full addresses, and officer names were already removed. The system will continue to protect privacy by using Scottsdale Fire's existing ChatGPT Teams account, which keeps all data within approved and secure systems.

For implementation, two options have been considered:

- API-based access, which is preferred because it is more secure and efficient.

- Local model hosting, which is possible but less practical due to high computer resource needs.

The AI reports will also include clear explanations of how results were created, so users can trust the process. Finally, the project will follow all organizational data handling rules and limit the storage of personal information as much as possible.

KEY REQUIREMENTS (SER-2):

Functional Requirements

1. The system shall allow authorized users to upload CSV or Excel files containing Fire and EMS data.

2. The system shall process and store at least 36 months of historical data, with the ability to append new monthly updates.

3. The system shall generate dashboards that visualize key performance indicators (KPIs) such as dispatch times, turnout times, travel times, and hospital transfer times.

4. The system shall allow users to select custom date ranges for KPI analysis.

5. The system shall generate PDF reports that summarize the data and insights.

6. The system shall provide plain-language explanations of data insights using AI tools.

Data Requirements

1. The system shall accept input files with the column structures provided by Scottsdale Fire (as outlined in the Fire and EMS datasets).
2. The system shall handle data files with a minimum of 50 columns and 24 months of historical records
3. The system shall ensure that new uploaded files can be appended to the existing dataset without overwriting historical data.

Security Requirements

1. The system shall operate using Scottsdale Fire's existing ChatGPT Teams account to ensure data privacy within approved environments.
2. The system shall restrict access to authorized users only.
3. The system shall exclude sensitive information such as GPS coordinates, full addresses, and officer names from all reports and dashboards.

Performance Requirements

1. The system shall generate dashboards from uploaded datasets within 30 seconds for files up to 100 MB in size.
2. The system shall generate PDF reports within 60 seconds after user request.

Usability Requirements

1. The system shall provide customizable dashboards allowing users to filter and sort KPIs by incident type, date range, station.
2. The system shall provide both visualizations (charts, graphs, maps) and p[lain-text summaries in reports.
3. The system shall provide transparent explanations of how the AI generated insights were created.

DELIVERABLES (SER-1):

- **Data Upload and Processing Tool**
    - **Value Provided:** Lets users upload Fire and EMS data in CSV/Excel format, checks the file structure, and adds new monthly data without losing past records.
    - **Dependencies:** Must be finished before dashboards and reports can work.
- **KPI Dashboard**
    - **Value Provided:** Shows key performance indicators (dispatch times, turnout times, travel times, hospital transfers) with filters for date, station, and incident type.
    - **Dependencies:** Requires the Data Upload and Processing Tool.
- **AI-Powered PDF Reports**
    - **Value Provided:** Creates PDF reports with charts, summaries, and plain-language AI explanations. Saves staff time and makes reports easy to share.
    - **Dependencies:** Needs both the Data Upload Tool and KPI Dashboard.
- **Security and Access Controls**
    - **Value Provided:** Protects data by using Scottsdale Fire's ChatGPT Teams account and role-based access. Ensures sensitive info like GPS and addresses stays out of reports.
    - **Dependencies:** Can be built alongside other pieces but must be in place before system release.
- **User Guides and Training Materials**
    - **Value Provided:** Provides clear instructions and examples so staff can upload data, use the dashboard, and generate reports with little training.
    - **Dependencies:** Written after main features are complete.
- **Project Documentation**
    - **Value Provided:** Includes meeting notes, user stories, and the project plan to keep the sponsor and team on the same page.
    - **Dependencies:** Produced and updated throughout the project.

ACRONYMS AND ABBREVIATIONS (ABET-3):

This section defines the technical acronyms and abbreviations used throughout the project plan to ensure clarity and consistency. Commonplace abbreviations are intentionally excluded. Terms are presented in their full definitions as they apply to this project's architecture, data workflows, security posture, and AI reporting components.

Acronym/Abbreviation	Definition
ABET-3	Course Outcome or ABET criteria tags are used to map sections to accreditation standards. These are internal labels used for grading and assessment.
AI	Artificial Intelligence. Used for automated insights, summarization and narrative generation in dashboards and reports.
API	Application Programming Interface. Used here for secure access to model services (ChatGPT Teams), dashboards, and PDF generation endpoints.
BI	Business Intelligence. Refers to data visualization/reporting tools and dashboards.
CSV	Comma-Separated Values. File format for data upload.
EMS	Emergency Medical Services. Refers to Scottsdale Fire's EMS operations data.
ETL	Extract, Transform, load. A common data pipeline pattern used for ingestion and cleaning of time-series data.
GIS	Geographic Information System. Mapping technology referenced in market analysis.
GPS	Global Positioning System. Sensitive data element removed before analysis.
KPI(s)	Key Performance Indicator(s). Metrics such as dispatch times, turnout times, and travel times visualized on dashboards.
LLM	Large Language Model. Refers to AI models, such as ChatGPT, used for narrative generation.

MVC	Model-View-Controller. Architectural pattern used for the web app.
PDF	Portable Document Format. Output format for AI-powered reports.
PII	Personally Identifiable Information. Sensitive information, like addresses or officer names, removed from the reports.
REST/Restful	Representational State Transfer. Architectural style for web services used in the project's API endpoints.
UI/UX	User Interface, User Experience. Refers to the front end dashboard and its usability.

DESIGN AND ARCHITECTURE

This project will adopt a modular, layered web architecture that is organized around MVC for the web app and a small data/analytics subsystem behind a clean service boundary. Each component in this system is replaceable via well-defined interfaces.

Top Level Components:

1. **Web UI (View)**
    a. This component will allow the user to upload CSV's, select from date ranges, choose analytics, view interactive charts and export PDF's with a Plain English narrative.
    b. This component will use REST/JSON so the front end can be swapped as long as they use this same contract.
2. **API Gateway (Controller)**
    a. This component will provide the single entry point for the UI and routes requests to internal services.
    b. This component will rely on REST endpoints such as "/data", "/export/pdf", etc.
3. **Authentication Service**
    a. This component is responsible for handling user authentication as well as authorizing access to datasets and exports.
    b. This component can integrate with a different Identity Provider using the Strategy and Adapter design pattern.
4. **Ingestion and Schema Service**
    a. This component handles the safe ingesting of the monthly CSV file upload and reconciliation with the existing dataset.

   b. Primarily, this will handle detecting and validating the CSV schema, providing friendly error reports for mismatching. It will also append logic to the internal Data Store without duplicating existing data. Additionally, data quality checks for nulls, overlapping ranges and other concerns will be handled here.

5. **Data Store (Metadata)**
   a. This component's purpose is to provide reliable storage optimized for time-bounded queries. Using curated fact tables which track events and records over time, indexes on time, station, category and others can be made for fast filtering. Metadata will also be stored for dataset versions, upload history and potential quality flags.
   b. This component is abstracted using a Repository interface such as Postgres or another Database without impacting any of the higher layers.
6. **Analytics Engine**
   a. This component will compute metrics and trends used by the dashboard and reports.
   b. This component will use KPI calculators, optimized for the Scottsdale Fire Department's metrics, to calculate trends. It will also employ time windowing, comparative analysis (by station, incident etc).
   c. This component will expose its algorithms via an AnalyticsService interface. This will also allow for new methods to be added using the Strategy pattern.
7. **Visualization Service**
   a. This component will turn the analytics provided by another component into a chart-ready series.
   b. This component will focus on normalizing the datasets and provide various options for visualization. These will serve as image snapshots for PDF's and data for Interactive UI. Additionally, the charting vendor can be swapped out for another via the Adapter pattern.
8. **Reporting and PDF Service**
   a. This component will generate the Plain English PDF of selected charts and metrics.
   b. This will employ a narrative generator with templated text and rules that summarize trends. Using AI such as ChatGPT, this generator will be expanded to include meaningful insights into each chart and highlight trends.
   c. The renderer/library for the PDFs will be pluggable using the Adapter pattern.
9. **Job Scheduler**
   a. This component will be used to handle/manage asynchronous and long-running tasks.
   b. This component will queue the different ingestion jobs, queries, and analytics generations. It will also have retry functionality, dead-letter handling and progress tracking for the UI.
10. **Audit and Observability**
   a. This component will offer users operational insight and different compliance logs.
   b. Specifically, audit logs will be kept of who uploaded files, when, what rows or fields were changed, etc. It will also allow for different metrics to be tracked and logged for viewing.

11. **External integrations (File storage, PDF engine)**
    a. This component acts as a boundary for outside systems to interact with internal services. Each external system will be accessed using the Adapter pattern to keep the core system itself decoupled. Some external systems that will be interacting with the core services are the file storage, pdf rendering services and possibly some email/share export services.

ALTERNATE DESIGN POSSIBILITIES (EM@FSE-B)

The project team considered one other main design choice: A scheduling roster agent for automating the Station scheduling process. This agent would have been implemented to allow Scottsdale Fire Department station staffing chiefs to automate the existing scheduling process at each fire station and vehicle. The project team has opted to keep this feature set as a potential future scope for planning, development and testing once the primary project has been completed.

IMPLEMENTATION STRATEGY

HIGH-LEVEL WORK BREAKDOWN STRUCTURE (SER-1):

Data Upload and Processing Tool:

- **Create File Upload Prompt -** End users will be able to select a file to upload from their personal file system. The prompt will confirm with the user that they have selected the desired file for processing. Provide the ability to exit the window with a cancel button.
- **Verify File Type -** Verify the file type is either CSV or Excel. Reject the file upload if not an accepted file type
- **Parse File -** Input file will be parsed for data and converted into an object that can be utilized for program functionality
- **Append Data to Existing Database** – Parsed data will be added to the already existing data. New data will be used in analytics and reports

  Skillset Requirement - Window creation and file manipulation

  Expected Time to Complete - Deliverable will take approximately 3 working days to complete. This should include implementation and functionality testing.

KPI Dashboard:

- **Create UI elements –** UI components that displays visual information and data cumulated from the AI analysis
- **Import necessary data** – Pull data from database into local memory to utilized by UI components
- **Configure each component** – Display priority information, each component will have a specify analytic to display.
- **Implement filter** – User will be able to decide between different categories and specifications for exact information that they are looking for.

  Skillset Required – UI/UX Design capabilities, data integration

  Expected Time to Complete – 5-7 working days (Approximately 46 hours)

AI-Powered PDF Reports:

- **Create PDF template –** Produce a template that can be easily filled with new and updated information
- **Import AI companion –** Establish AI connection
- **Analyze data –** Provide AI with data received from the user CSV file
- **Parse AI response -** AI response will be converted to an object that can be imported into the PDF
- **Export data in PDF format -** PDF File will be created with the analytics generated from the AI companion

Expected Time to Complete - 3-4 working days (Approximately 27 hours)

Security and Access Controls:

- **Setup Microsoft Graph API -** Implement the Microsoft Graph API in order to pull information from Microsoft Teams
- **Establish secure connection -** Utilize the credentials provided by our Sponsor to create a secure login connection

Expected Time to Complete - 1 working day (Approximately 7 hours)

User Guides and Training Materials:

- **Create procedures and usage examples**
- **Format Material into PDF documents**

Expected Time to Complete - 1 working day (Approximately 4 hours)

SCHEDULE / TIMELINE (SER-1):

- **October 1 – October 11**: Finalize project plan, complete Data Upload and Processing Tool deliverable
- **October 13 – November 15**: KPI Dashboard
- **November 18 – December 9**: AI-Powered PDF Reports, discuss expectations for the start of next semester
- **January 19 – February 7**: Security and Access Controls, Beta testing
- **February 9 – February 21**: User Guides and Training Materials, Apply findings from testing.
- **February 23 – May 1**: Finalize project/system. Distribute to customers

REQUIRED HARDWARE (SER-1, EM@FSE-o):

- Desktop computer with network capabilities

THIRD PARTY CONTENT (SER-1, EM@FSE-o):

- Azure, must have the ability to add the application to the Azure portal in order to utilize Microsoft Graph API
- Microsoft Teams Access, the customer will provide a login for the team to utilize in order to receive Fire and EMS data.

QUALITY (SER-2):

The project's quality goals focus on delivering a robust dashboard system that accurately visualizes KPIs such as call processing times, turnout times, and travel times from uploaded CSV/Excel files, generates reliable AI-driven PDF summaries, and ensures data security and transparency in AI processes. Key metrics include:

- **Functional Quality Metrics**: Accuracy of KPI visualizations and AI summaries, measured by error rates (target: <5% discrepancy between input data and output interpretations, validated through sample datasets); completeness of PDF reports, assessed by coverage of all specified metrics (100% inclusion of fire and EMS department data elements like historical trends over 24-36 months).
- **Non-Functional Quality Metrics**: System response time for data ingestion and visualization (target: <10 seconds for processing monthly updates with ~50 columns); data security compliance (zero breaches in handling sensitive information, audited via access logs); AI transparency (100% of outputs include optional explanations of reasoning); usability (user satisfaction score >80% from sponsor feedback on customizable formats, such as text vs. visual explanations).

To meet these metrics, our team will employ a comprehensive test strategy addressing both functional and non-functional aspects:

- **Unit and Integration Testing**: Use automated testing frameworks appropriate to the chosen development language and tools to cover edge cases such as large datasets, or redacted sensitive columns such as addresses and GPS coordinates.
- **End-to-End Testing**: Simulate real workflows, such as appending monthly Excel updates to historical data, to verify overall system fidelity against sponsor-provided templates and BI examples.
- **Security and Ethical Reviews**: Conduct regular code audits and penetration testing to ensure private data handling via API (Scottsdale Fire's ChatGPT Teams account) or local models, aligning with sponsor concerns about data storage.
- **Code Style and Reusability**: Adhere to standard coding conventions for the selected language to enhance readability, with modular design to support reusability. Also, generate HTML files for creating API documentation that describes classes, methods, parameters, return values, and usage which allows an interface reference for the codebase. incorporate inline comments for complex code snippets, so white box testing has an easier time addressing bugs. The team will use modeling diagrams such as use case, activity, and class diagrams as quality control artifacts to enhance system transparency against sponsor requests for software flow.

REFERENCES/SOURCES OF INFORMATION (EM@FSE-Q)

This project has four key research areas:

1. **Data Visualization**
    a. This area is immensely important. The project team seeks to understand best practices for visualizing time-series and categorical data, specifically in the context of public safety metrics. Research in this area will ensure that the charts, dashboard and PDF reports communicate trends clearly and support the Scottsdale Fire Department making informed decisions.
    b. References:
        i. Apache ECharts open source charting library: https://echarts.apache.org/en/index.html
        ii. Data.gov catalog of open source data tools: https://resources.data.gov/categories/data-tools
        iii. Google Charts gallery for examples and guidance: https://developers.google.com/chart/interactive/docs/gallery
2. **Web Application Architecture and Modular Design**
    a. Research in this area will allow for the project team to efficiently develop the platform using a layered, modular approach. Understanding how the modern web app architecture works will help ensure scalability, security and maintainability.
    b. References:
        i. MDN Server-side website programming web documents: https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Server-side
        ii. Microsoft Microservices architecture guide: https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/microservices
        iii. OWASP Web Security testing guide: https://owasp.org/www-project-web-security-testing-guide
3. **CSV Data Ingestion, Data Cleaning and Time-Series Storage**
    a. This research area is fundamental to the project scope. The platform will need to be capable of ingesting large monthly CSVs and maintaining 36 months+ of time-series data. This will require extensive knowledge of the ETL (Extract-Transform-Load) pattern, schema validation and time-series indexing practices to ensure reliable and efficient data processing.
    b. References:
        i. Python web docs on reading and writing CSV files: https://docs.python.org/3/library/csv.html
        ii. Web guide for building ETL pattern programming: https://visual-flow.com/blog/building-an-etl-design-pattern-the-essential-steps
        iii. InfluxDB web docs, an open source time-series database with guides on schema design, ingestion, and querying: https://docs.influxdata.com/influxdb/v2
4. **Automated Reporting and PDF Generation**

a. Dedicating time and resources into this focus area is important as the platform must generate PDF reports with both charts and plain English narratives. Thorough understanding of the different PDF generation libraries and automated reporting frameworks will help to produce high-quality, accessible outputs.
b. References:
    i. ReportLab user guide, an industry-standard Python PDF generation library: https://www.reportlab.com/documentation/
    ii. Web guide for automating Python PDF generation: https://realpython.com/pdf-python

SCALABILITY (EM@FSE-J)

The project is designed for scalability to handle growing data volumes and potential expanded use in fire and EMS contexts, such as appending monthly updates with ~50 columns and extending historical datasets beyond 24-36 months. Scaling and sustainability will be achieved through:

- **Scaling Mechanisms**: Leverage modular architecture with API integrations (e.g., sponsor's ChatGPT Teams account or ASU AI licenses) for efficient data handling without heavy storage needs. Potentially use open-source tools like Grafana for visualizations, which can be distributed across local or low-cost hosted environments to manage increased loads, such as multiple users querying KPIs simultaneously.
- **Sustainability Elements**:
    - **Revenue Streams**: For long-term viability, consider open-sourcing the dashboard to encourage community contributions from other public safety organizations, or seek grants/ASU funding to cover minimal maintenance.
    - **Key Partners**: Collaborate with Scottsdale Fire for ongoing data/API access and ASU for tool licenses/expertise, supporting feature additions like scheduling without high costs.
    - **Costs**: Prioritize cost-effective options, such as local models or API hosting to avoid expensive alternatives; automate workflows for data ingestion to reduce operational expenses.
    - **Key Resources**: Sponsor-provided data dictionary, Excel templates, and BI examples; team-shared Google Docs for collaborative development.

Anticipated constraints include computational limits such as local models handling only ~100 concurrent users due to AI processing demands, and storage growth from accumulating historical data. Mitigations involve:

- Caching, partitioning, and optimization techniques to handle larger datasets efficiently.
- Containerization (e.g., Docker) for flexible deployment, allowing easy shifts to scalable hosting if needed without initial high costs.
- Real-time monitoring integrated into the chosen visualization platform for resource alerts, enabling proactive adjustments.

This approach aims to grow the system sustainably within the two-semester scope, focusing on low-cost, secure expansion aligned with sponsor preferences.

OTHER SPECIAL CONSIDERATIONS (ABET-2):

This project handles sensitive fire and EMS data (~50 columns, 24-36 months history), requiring prompt paperwork completion for access and redaction of elements like addresses/GPS to balance privacy with analytics—use APIs for secure handling where possible. Emphasize ethical AI: Provide options for users to view explanations as to why/how returned data arrives at it's answers (text/visual, straightforward per sponsor preference) to maintain transparency without excess details.

Key dependencies:
Receive sponsor files (templates, Excel columns, BI visuals, data dictionary), to inform user stories/epics. Consider tools like Grafana for visualizing uploaded Excel data and integrating with databases/LLMs for Plain english explanations, or Docker for modular deployment to enhance portability. Foster team Collaboration through a shared Google Docs.

PROCESS

Process Description and justification (SER-1)

We plan to use an Agile development process with two-week sprints and continuous integration. This method is appropriate because the sponsor will provide data samples and feedback throughout the project. Since the team will need to make changes to the machine learning and language model components, it is important to have regular sponsor review. The project also involves building interactive user interfaces and data ingestion features, so delivering these in stages and showing progress will be beneficial. Adaptations for this project: We will follow a two-week sprint schedule. Each sprint will start with planning, may include a check-in during the sprint if needed, and will finish with a demonstration for the sponsor. For backlog refinement, we will keep a sprint-level backlog that organizes user stories and epics by priority. Initially, we will define these using templates provided by the sponsor and notes from our meetings. We will keep documentation simple by using a shared Google Doc and the repository README. We will update the data dictionary whenever we receive new data from the sponsor. Before adding sponsor data to any hosted service, we will use a checklist to ensure the data is redacted, access credentials are approved, and the sponsor has given their approval. We will reserve time in each sprint for technical spike tasks. For example, we might compare using a local language model to using an API, or test an integration with Grafana. These spike tasks help us reduce uncertainty. The sponsor expects to see results on a regular basis and may ask for different output formats. By using Agile, we can deliver our work in small increments and get regular feedback from the sponsor, which helps us meet these expectations. The two-week sprint cycle gives us enough time to test data processing and AI outputs while keeping our progress steady.

This process will work for the team because it allows for development and continuous sponsor engagement, which is necessary for a project involving data visualization and AI components where requirements may evolve based on sponsor feedback and data availability.

Tools (SER-1, EM@FSE-O):

Non-hardware tools the team will use (primary tools and purpose): **GitHub** (private repo) — source control, issues, PRs, CI. (Branch protection + PR reviews required.) **Google Drive / Google Docs** (shared folder) — living project plan, meeting minutes, sponsor-facing documents. (Mike will upload the template and maintain it.) **Microsoft Teams / Slack** — day-to-day communication and sponsor/Frank quick questions (Frank suggested ChatGPT Teams; use Teams when sponsor requires). **Jupyter / Python** (pandas, numpy) — data cleaning, exploratory analysis, and unit tests for ETL. **Node.js / Express + React** — backend API and frontend dashboard stack (prototyping choices; final choice documented in the architecture section). **Docker** — containerized development and deployment (ensures reproducible environments for local model experimentation if needed). **Grafana** (or equivalent BI library) — visualization for KPI dashboards (used for prototypes; select open-source stack to avoid high licensing). **LLM API access** (sponsor-provided ChatGPT Teams or ASU licenses) — AI summarization & explanation generation for PDF reports (primary approach to avoid storing sensitive data). **wkhtmltopdf / Puppeteer or PDF generation library** — compile dashboard text/visual summaries into downloadable PDF reports. **Figma or similar** (optional) — UI mockups and design reviews (if needed). **Issue tracker + project board** (GitHub Projects) — sprint board, backlog, epics, and user stories. **Security tooling**: simple static analysis (ESLint, bandit-like checks for Python), dependency scanning (Dependabot), and logging/audit tools for access tracking.

Relevant external links will be provided in the final documentation with labeled link text rather than raw URLs.

Roles and Responsibilities (SER-1):

We will define each team member's role clearly, but we also expect everyone to be flexible. While each person will have a primary responsibility, all team members are expected to contribute to the codebase when needed. Core roles (role, primary responsibilities, suggested owner): **Project Manager / Scrum Lead** — coordinate sprints, maintain backlog, run sprint ceremonies, deliver milestone artifacts. Suggested owner: Mike. **Sponsor Liaison / Data Sponsor Contact** — primary contact to the sponsor, ensures data access, clarifies domain semantics, and approvals for release. Suggested owner: Frank (sponsor) with team point: Mike. **Data Engineer** — ingest and clean sponsor Excel/CSV files, maintain ETL pipeline, implement redaction and schema handling; maintain data dictionary. Suggested owners: Zachary (data-focused) / Frank for domain checks. **Backend Lead** — design and implement APIs, data storage, authentication, and PDF-generation endpoints. Suggested owner: Zachary / volunteer. **Frontend Lead (UI/UX)** — dashboard design, KPI visualization integration, user interaction, and export functionality. Suggested owner: Andrew. **ML/LLM Engineer** — integrate LLM APIs, design prompts, validate AI explanations, evaluate local model feasibility, and test the transparency of outputs. Suggested owners: team rotation or Mike and Zachary. **QA & Security Lead** — define test cases, perform end-to-end testing, run basic pentests for data endpoints, and validate compliance with redaction rules. Suggested owner: Andrew or rotating role. **Documentation & Demo Owner** — ensure project artifacts (meeting minutes, plan updates, README, user stories) are up to date and prepare demo materials for sponsor reviews. Suggested owner: Mike / Andrew. For certain responsibilities, such as QA and Documentation, we will rotate the owner each sprint. This approach will help everyone gain experience and ensure that there is always backup for each role. We will schedule these rotations during sprint planning and record them for each sprint. Every team member is expected to contribute

code or tests, participate in at least one sponsor demo, and make sure their work is linked to an issue or user story in the tracker.

Team members must be involved throughout the project's execution, but should be assigned a specific role to take ownership over parts of the project. Every team member, regardless of role, must contribute significantly to development of the software deliverables.

Location of Project Artifacts (SER-1):

We will organize our project artifacts in a way that balances usability, reliability, and effective change management. **Source code and CI**: Private GitHub repository (main branch protected; use feature branches, PR reviews, and tags/releases for milestones). Store automated test artifacts and CI logs in GitHub Actions. **Project documents & meeting artifacts**: Google Drive — Shared > SER401 > [Project Name] folder. Subfolders: ProjectPlan/, MeetingMinutes/, DataSamples/ (redacted), Design/. Mike will maintain the ProjectPlan folder and upload the final Plan Review artifacts. **Data samples & data dictionary**: Sponsor-provided samples stored in Google Drive > DataSamples/ (redacted copies for team use) and mirrored in the repo data/samples/ for reproducible tests (only non-sensitive samples committed). Access to raw sponsor files is controlled — they will not be committed to the public repo. **Design assets**: Figma files (if used) or Design/ folder in Drive. **Backups / Releases**: Major deliverables zipped and added to a Releases/ folder in Drive and as GitHub release assets. **Access control**: Drive permissions are restricted to the team and sponsor (exception of ASU TA), with the GitHub repository under the team organization and limited to team members. Use branch protection rules and require PR reviews before merging. This setup allows the sponsor to access deliverables in Google Drive, ensures strict version control and audit trails for code in GitHub, and supports change management through pull requests and release tags.

This organization supports usability through clear folder structures, reliability through multiple backup locations and version control, and enables effective change management through pull requests and release tracking.

Sponsor communications (ABEt-3):

Agreed communication plan (must be aligned with sponsor; the team will confirm schedule with Frank): Each week, the Project Manager will send a written update by email or Google Doc. This update will be a short status summary each Friday, including 2 to 4 bullet points about progress, blockers, and upcoming tasks. We will use concise language, since the sponsor prefers straightforward answers. At the end of each two-week sprint, we will hold a 30 to 45 minute demo and feedback session with Frank or a designated sponsor representative. During these demos, we will show working features such as data ingestion, dashboard visuals, and PDF summary examples, as well as discuss proposed priorities for the next sprint. For ad-hoc technical check-ins, we will use Microsoft Teams to ask quick questions or exchange example files. Since Frank has offered API access and sample files, Teams will be our preferred channel for quick communication. Forty-eight hours before each demo, the team will send a one-page demo brief along with the data samples used for the demo (with sensitive information redacted) so the sponsor can prepare in advance. Before moving any sponsor data into shared staging or production systems, we will require explicit sponsor approval by email. If we encounter blockers that could threaten milestone delivery, we will send an immediate email and schedule a special Teams call within 48 hours to address the issue.

Meeting cadence summary:

Sprint planning — every 2 weeks (team only; invite sponsor on request)

Demo / sponsor review — every 2 weeks aligned to sprint end (team + sponsor)

Weekly status email — every Friday (team → sponsor)

Ad-hoc — via Teams for quick exchanges and sample delivery

This information has been agreed upon with the project sponsor and provides multiple communication channels to ensure effective project coordination and feedback.

Risk management

identified Potential risks (SER-2):

Below are the identified risks along with their impact and likelihood:

- **Sponsor data delay / incomplete data**
  - Impact: High — inability to build accurate dashboards or write realistic user stories.
  - Incidence: Medium (meeting notes indicate Frank will provide files, but availability may vary).
- **Sensitive data / privacy concerns (addresses, GPS, PII)**
  - Impact: High — legal and ethical impact; sponsor will not allow sensitive data into public systems.
  - Incidence: Medium.
- **LLM inaccuracies / hallucinations in generated summaries**
  - Impact: Medium–High — sponsor requires transparent and accurate explanations; erroneous AI outputs may reduce trust.
  - Incidence: Medium.
- **API access limitations or quota (sponsor-provided ChatGPT Teams access)**
  - Impact: Medium — rate limits could block automated PDF generation or batch summarization.
  - Incidence: Medium.
- **Computational resource constraints for local model experiments**
  - Impact: Medium — local LLMs may require GPUs/compute that the team does not have; slows or blocks feasibility tests.
  - Incidence: Low–Medium.
- **Scope creep (new BI features like scheduling, GIS layers)**

        - Impact: Medium — increases schedule and complexity beyond the two-semester plan.
        - Incidence: High (meeting notes already mention optional future scope).
    - **Team member availability / coordination problems**
        - Impact: Medium — misses sprint commitments, reduces delivery velocity.
        - Incidence: Medium.
    - **Integration issues with sponsor systems / data formats**
        - Impact: Medium — additional ETL effort, rework.
        - Incidence: Medium.
    - **Third-party cost/licensing (commercial BI too expensive)**
        - Impact: Medium — may force shift to open-source solutions and additional dev time.
        - Incidence: Low–Medium (e.g., a cost of $300k is prohibitive and has already been noted).
    - **Security breach or improper handling of sponsor data**
        - Impact: High — reputational and contractual risk.
        - Incidence: Low if access control is followed, but non-zero.
    - **Timeline / milestone slippage (missing Oct 11 working version or Plan Review 1)**
        - Impact: High (missed academic deadlines).
        - Incidence: Medium.
    - **Regulatory / contractual constraints (e.g., PHI, local policies)**
        - Impact: High if present.
        - Incidence: Low (depends on sponsor data specifics).

Potential risks have been considered for all aspects of the project, including group dynamics, sponsor communication, infrastructure, design limitations, implementation blockers, and incomplete requirements.

MITIGATION STRATEGIES (SER-2):

For each of the risks identified, the following approaches will be used to mitigate them:

- **Sponsor data delay / incomplete data**
  - Mitigation: Create mock datasets that reflect the expected schema early, based on meeting details (~50 columns, 24–36 months). Start development using mocks to enable work to continue in parallel. Schedule a concrete deadline with the sponsor for the initial sample (confirm in writing). If samples are late, use mock data sprint deliverables to validate ETL & dashboard flows.
  - Cost/impact: Low developer time to create mocks (1–2 person-days); reduces schedule risk.
- **Sensitive data / privacy concerns**
  - Mitigation: Implement a data redaction checklist and require the sponsor to provide redacted samples for general dev use. Use API-only LLM approach (sponsor-provided Teams API) with no persistent storage of raw PII in project systems. Encrypt sensitive files at rest and restrict access to the Drive and repository.
  - Cost: Minor configuration + process time. If the sponsor requires full non-redacted data, additional legal/IRB/approval steps may be required (time cost: weeks).
- **LLM inaccuracies / hallucinations**
  - Mitigation: Use structured prompt templates and verification steps: for each AI-generated summary, create a small rule-based or unit test check (e.g., totals, date ranges) to catch obvious inconsistencies. Provide human-in-the-loop review for initial deliverables (sponsor review step). Offer configurable verbosity and an "explain my answer" option to improve transparency and allow the sponsor to see the reasoning.
  - Cost: Extra QA time per report (0.5–1 person-day per sprint until the report is stable).
- **API access limitations or quota**
  - Mitigation: Track usage and implement request batching and backoff strategies; cache repeated queries to improve performance. If quotas are too low, fall back to scheduled processing (generate PDFs overnight) or evaluate ASU-provided licenses.
  - Cost: Development time to implement batching, plus possible additional costs if the sponsor requires an expanded quota.
- **Computational resource constraints for local models**
  - Mitigation: Prioritize API integration as the primary approach; treat local models as optional research spikes. If a local model is required later, consider using cloud spot instances (cost-effective) or reducing the model size and utilizing quantized models.
  - Cost: Cloud GPU usage if chosen (estimated $50–200 per day, depending on instance and duration). Plan only if the sponsor requires an offline model.
- **Scope creep**
  - Mitigation: Maintain a clear backlog and a scope change request process with the sponsor (any new major features require re-prioritization and schedule rework). Reserve buffer in timeline (one sprint) for additional features.
  - Cost: A scope change will shift the timeline; agreeing on priority reduces unexpected slippage.
- **Team member availability / coordination**

    - Mitigation: Cross-train (pair programming) and require each story to have a secondary owner. Use explicit sprint commitments and daily asynchronous status via the project board.
    - Cost: Small time investment for cross-training (1–3 person-days early).
- **Integration issues with sponsor systems / data formats**
    - Mitigation: Define the data schema upfront and request a data dictionary from the sponsor (Frank committed to provide this). Build ETL with flexible schema mapping and unit tests. Early spike to validate file formats with provided sample files.
    - Cost: Short spike (2–4 person-days) to validate and build robust parsers.
- **Third-party cost/licensing**
    - Mitigation: Prefer open-source BI and visualization libraries (Grafana, Plotly, D3) for the prototype. If a paid tool is essential, include licensing in a cost proposal and treat it as out-of-scope until sponsor funds are allocated.
    - Cost: Open-source tooling reduces costs but increases dev time.
- **Security breach**
    - Mitigation: Enforce minimal privilege access to Drive and the repository, use MFA, and remove sensitive files from the repository. Conduct a lightweight security review and document procedures for data handling.
    - Cost: Low (configuration + one security review session); high if a real breach occurs.
- **Timeline / milestone slippage**
    - Mitigation: Milestone decomposition with clear owners; monitor progress in each sprint and flag slippage early. Maintain a contingency buffer in the schedule for high-risk tasks (e.g., a 1-sprint buffer before the Oct 11 milestone).
    - Cost: May require prioritizing higher-value features over lower-value ones.
- **Regulatory / contractual constraints**
    - Mitigation: Conduct early sponsor discussions to identify any regulatory constraints and incorporate required redactions or compliance steps into the project plan. If required, consult with the sponsor's legal/compliance teams and document any constraints.
    - Cost: Time for approvals and possibly restricted functionality until compliance is achieved.
- **Summary of highest-priority mitigations (immediately actionable):**
    - Request a redacted data sample and data dictionary from the sponsor ASAP (Frank indicated that these will be provided). If not provided by the agreed date, use mock datasets.
    - Start sprint 0 using mocked sample data and build ETL & dashboard skeleton to reduce delivery risk.
    - Use sponsor API (ChatGPT Teams / ASU license) as primary LLM approach; only spike local model if sponsor insists.
    - Apply basic access control to Drive and the repository, and adopt the data redaction checklist before committing any real data.

All identified risks have mitigation strategies listed, including discussion of risks that cannot be mitigated fully. Timeline buffers and other passive mitigation strategies are demonstrated throughout other sections of this document.